

Exposé Informatik-Projekt

Vincent Trinz | 2D-Plattformer »Pillarfall« | Projektstart: 06.05.2026

Vom 06.05. bis 10.05.2026 begann die Arbeit an unserem Projekt mit einer Recherchephase. Da ich vorher nur einfache Java-Projekte ohne zusätzliche Bibliotheken programmiert hatte, waren libGDX und Gradle komplett neu für mich. Deshalb schaute ich mir zuerst die offizielle Dokumentation von libGDX und mehrere YouTube-Tutorials an, um zu verstehen, wie ein Projekt aufgebaut ist. Am Anfang hatte ich vor allem Probleme mit Gradle. Ich wusste nicht genau, warum Bibliotheken in der Datei *build.gradle* eingetragen werden müssen oder wofür die Module *core* und *lwjgl3* zuständig sind. Um das besser zu verstehen, ließ ich mir von ChatGPT die Projektstruktur anhand eines einfachen Beispiels erklären. Dadurch konnte ich den Aufbau des Projekts deutlich besser nachvollziehen.

Am 06.05.2026 fand außerdem die erste Besprechung mit meinem Projektpartner Abdullah statt. Gemeinsam entschieden wir uns für einen 2D-Plattformer mit dem Namen „Pillarfall“ und erstellten ein erstes grobes Klassendiagramm. Abdullah richtete das GitHub-Repository ein. Danach teilten wir die Aufgaben auf: Ich übernahm die Tile-Klasse, die Enemy-Klasse sowie später das Health- und Attack-System. Abdullah kümmerte sich um das Rendering, die Kamera und die Kollisionslogik.

Vom 07.05. bis 10.05.2026 begann ich mit der Programmierung der Tile-Klasse und der Enemy-Klasse. Zuerst erstellte ich bei beiden Klassen nur den Grundaufbau mit den wichtigsten Attributen, einem Konstruktor und leeren Methoden. Dabei traten schon die ersten Probleme auf. Ich wusste anfangs nicht, welche libGDX-Klassen ich importieren musste und in welchen Paketen sie sich befinden. Außerdem war die Java-Paketstruktur für mich neu, da ich bisher nur kleinere Projekte ohne Pakete programmiert hatte. Deshalb ließ ich mir von ChatGPT erklären, welche libGDX-Pakete für Texturen, Sprites und mathematische Berechnungen zuständig sind. Dadurch konnte ich mich schneller in die neue Projektstruktur einarbeiten.

Vom 11.05. bis 24.05.2026 arbeitete ich am Health-System für den Spieler. In der ersten Version konnte der Spieler maximal 150 Lebenspunkte haben. Dafür programmierte ich einen aktuellen Lebenspunktwert sowie die Methoden zum Erhalten von Schaden, zum Heilen und zur Überprüfung, ob der Spieler besiegt ist. Ein Herz steht dabei für 50 Lebenspunkte. Während der Entwicklung gab es ein Problem: Ein Herz wurde schon als leer angezeigt, obwohl der Spieler noch fast alle Lebenspunkte hatte. Ich fragte Claude AI nach Hilfe. Dort wurde mir erklärt, wie die Schwellenwerte richtig berechnet werden müssen. Danach konnte ich den Fehler schnell beheben.

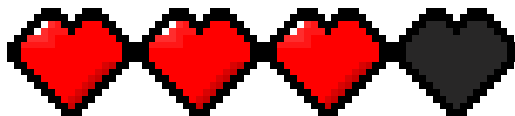


Abb. 1: Volles und leeres Herz aus dem Health-System des Spielers (Heart.png / Heart_empty.png).

Vom 24.05. bis 26.05.2026 programmierte ich das Attack-System für den Spieler. Dabei legte ich die Angriffsreichweite, den Schaden und einen Cooldown fest, damit der Spieler nicht ununterbrochen angreifen kann. Dabei trat ein Problem mit dem Cooldown auf. Zuerst erhöhte ich den Timer bei jedem Bild um einen festen Wert. Dadurch lief der Cooldown je nach Bildrate unterschiedlich schnell. Bei meiner Recherche fand ich in der offiziellen libGDX-Dokumentation und auf Stack Overflow^{*1} heraus, dass man dafür `getDeltaTime()` verwenden sollte. Diese Methode gibt die tatsächlich vergangene Zeit zwischen zwei Bildern zurück. Nachdem ich den Timer darauf umgestellt hatte, funktionierte der Cooldown auf allen Geräten gleich und ohne Probleme.

Vom 26.05. bis 28.05.2026 versuchte ich, die Kollisionslogik für die Spielwelt zu programmieren. Mein Ziel war es, feste Blöcke in der TiledMap zu erkennen, damit der Spieler nicht durch sie hindurchlaufen

*1: <https://libgdx.com/wiki/graphics/graphics>

oder fallen kann. Der Code ließ sich zwar ohne Fehler starten, trotzdem fiel der Spieler immer durch alle Blöcke. Ich versuchte den Fehler mehrere Stunden lang mit `System.out.println()` zu finden, hatte aber keinen Erfolg. Später stellte sich heraus, dass ich den falschen Layer verwendet und außerdem den Layer-Namen falsch geschrieben hatte. Dadurch konnte das Programm die Kollisionsdaten nicht finden. Da ich das Problem auch nach mehreren Tagen nicht lösen konnte, übernahm mein Projektpartner Abdullah diese Aufgabe.

Vom 28.05. bis 31.05.2026 programmierte ich den ersten Gegner. Dafür legte ich eine Startposition, eine Textur, eine Geschwindigkeit und eine einfache KI fest, damit sich der Gegner auf den Spieler zubewegt. Am Anfang erschien der Gegner an der falschen Stelle. Der Grund war, dass das Koordinatensystem in libGDX anders funktioniert als ich erwartet hatte. Nach einem Hinweis von meinem Projektpartner Abdullah und einer kurzen Recherche in der Dokumentation konnte ich den Fehler beheben. Auch das richtige Laden und Anpassen der Größe der Gegnergrafik war anfangs neu für mich. Mithilfe der Dokumentation und eines Beispiels von Claude AI konnte ich den Sprite richtig einbinden und skalieren.



Abb. 2: Spieler-Sprite (Peter.png) und Gegner-Sprite (Enemy.png) nach der Implementierung.

Am 31.05. bis 10.06.2026 erweiterte ich den Gegner um ein eigenes Health- und Attack-System. Analog zum Spieler erhält der Gegner die Methoden `damage()` und `isDead()` sowie einen eigenen Cooldown. Beim Testen fiel mir auf, dass der Gegner den Spieler auch nach dessen Tod weiter angriff, da ich vergessen hatte, `isDead()` abzufragen. Diesen Fehler behob ich, indem das Spiel jetzt in jedem Frame mit `isDead()` prüft, ob der Spieler noch lebt, und beim Tod sofort zum Game-Over-Screen wechselt. Ich passte `ATTACK_DAMAGE` und `ATTACK_RANGE` mehrfach an und nutzte Claude AI, um Richtwerte für Schadenswerte in einem 2D-Plattformer mit drei Herzen zu erfragen.

Vom 10.06. bis 25.06.2026 überarbeitete ich das Health-System. Nach mehreren Spieltests fiel mir auf, dass 150 maximale Lebenspunkte bei einem Schaden von 25 pro Treffer unpraktisch waren. Der Spieler brauchte sechs Treffer, bis er besiegt war, was sich mit der Herzanzeige nur schwer erkennen ließ. Deshalb entschied ich mich nach einer Diskussion mit ChatGPT, `MAX_HEALTH` auf 100 zu reduzieren. Außerdem sollte jedes Herz jetzt 33 Lebenspunkte darstellen. Zusätzlich wollte ich halbe Herzen einführen, damit auch kleinerer Schaden besser angezeigt wird. Ich fragte Claude AI, wie die Berechnung für volle, halbe und leere Herzen funktionieren kann. Dabei wurde mir erklärt, dass zuerst berechnet wird, wie viele halbe Herzen der Spieler noch hat. Dafür werden die Lebenspunkte durch 16,5 geteilt und aufgerundet, da ein halbes Herz 16,5 Lebenspunkten entspricht. Danach wird für jedes Herz geprüft, ob es voll, halb oder leer angezeigt wird. Eine eigene Textur für das halbe Herz brauchte ich dabei nicht: Es wird einfach nur die linke Hälfte des vollen Herzens gezeichnet. Dafür musste ich die Methode `renderHealthBar()` anpassen. Mit der Erklärung von Claude AI konnte ich die Änderungen ohne größere Probleme umsetzen.

Vom 25.06. bis 30.06.2026 arbeitete ich an weiteren Verbesserungen und beseitigte verschiedene Fehler im Spiel. Ich änderte die Spawn-Logik der Gegner, sodass ihre Positionen nicht mehr direkt im Code festgelegt werden, sondern über eine Liste in der World-Klasse verwaltet werden. Dadurch lässt sich das Spiel später einfacher erweitern. Zum Schluss fiel mir noch auf, dass sich die Herzanzeige bei

*1: <https://libgdx.com/wiki/graphics/graphics>

mehreren schnellen Treffern manchmal erst einen Frame später aktualisierte. Ich löste das Problem, indem ich den Lebensstand in jedem Frame direkt neu berechnen ließ, anstatt den alten Wert zwischenspeichern.

Am 30.06.2026 versuchte ich, den Spieler durch ein neues Sprite zu ersetzen. Das bisherige Peter.png war nur ein Platzhalter. Deshalb lud ich mehrere neue Sprites herunter und versuchte, sie ins Spiel einzubauen. Dabei trat jedoch ein Problem auf: Das neue Sprite hatte eine Auflösung von 64×64 Pixel statt 32×32 Pixel. Dadurch wurde der Spieler im Spiel zu groß dargestellt und die Kollisionsbox passte nicht mehr zur Figur. Der Spieler lief teilweise durch Blöcke oder blieb an unsichtbaren Hindernissen hängen. Ich versuchte, den Fehler mit setSize() sowie durch Änderungen der WIDTH- und HEIGHT-Konstanten in der Player-Klasse zu beheben. Das verbesserte die Darstellung jedoch nicht, sondern verschob die Kollisionsbox sogar noch weiter. Auch Claude AI konnte mir nur allgemeine Tipps zur Skalierung von Texturen geben, wodurch das eigentliche Problem nicht gelöst wurde. Da ich bis zum Abgabetermin keine funktionierende Lösung fand, blieb Peter.png als Spieler-Sprite im Spiel.

Vom 01.07. bis 06.07.2026 programmierte ich den Game-Over-Screen und das Hauptmenü. Das Hauptmenü enthält die Schaltflächen „Spielen“, „Einstellungen“ und „Verlassen“. Im Game-Over-Screen kann der Spieler „Restarten“, „Hauptmenü“ oder „Verlassen“ auswählen. Da ich vorher noch nie Benutzeroberflächen mit libGDX erstellt hatte, war das für mich neu. Das größte Problem war die Erkennung der Mausklicks. Als ich die Klicks zuerst selbst abfragen wollte, wurden sie an einer anderen Position erkannt, obwohl die Buttons an der richtigen Stelle angezeigt wurden. Der Grund dafür ist, dass die Maus- und Spielkoordinaten unterschiedlich aufgebaut sind. Die Maus zählt die Y-Koordinate von oben nach unten, das Spiel jedoch von unten nach oben. Durch die libGDX-Dokumentation sowie Hinweise von ChatGPT und Claude AI fand ich heraus, dass es dafür in libGDX die Klassen Stage und ClickListener gibt. Diese übernehmen die Umrechnung der Mausposition automatisch. Nachdem ich die Buttons damit umgesetzt hatte, funktionierten sie wie erwartet.

Am 06.07.2026 wurden alle implementierten Features zusammengeführt und das Projekt für die Abgabe vorbereitet. Nicht fertiggestellt blieben das neue Spieler-Sprite, Animationen, zusätzliche Gegner-Typen sowie eine vollständig ausgearbeitete Spielzone. Das Projekt demonstriert dennoch die grundlegenden Mechaniken eines 2D-Plattformers: Spielerbewegung, Physik, Kollision, Kampfsystem, überarbeitetes Health-System mit halben Herzen (100 HP, 33 HP/Herz) und Menüführung. Rückblickend war das Einarbeiten und Nutzen in bzw. von libGDX die größte Herausforderung. Die Kombination aus offizieller libGDX-Dokumentation, Stack Overflow, YouTube-Tutorials und KI hat dabei geholfen, die meisten Hindernisse zu verstehen und zu überwinden.

Verwendete KI-Unterstützung (ChatGPT und Claude AI)

Während des Projekts nutzte ich ChatGPT und Claude AI regelmäßig zur Unterstützung. Die KI half mir dabei, neue Themen besser zu verstehen, Fehler zu finden und Lösungsansätze für verschiedene Implementierungsprobleme zu entwickeln.

Zu den wichtigsten Einsatzbereichen gehörten:

- Erklärung der Projektstruktur von libGDX und Gradle
- Hilfe bei Imports und der Verwendung von libGDX-Klassen
- Unterstützung beim Health- und Attack-System
- Fehlersuche bei Kollisionen und Gegnern
- Erklärungen zur Berechnung von Timern mit getDeltaTime()
- Unterstützung bei der Erstellung der Benutzeroberfläche (UI), z. B. Hauptmenü und Game-Over-Screen
- Hilfe bei der Erkennung von Mausklicks und der Positionierung von UI-Elementen

*1: <https://libgdx.com/wiki/graphics/graphics>

- Tipps zur Skalierung von Sprites und zur Anpassung der Kollisionsbox

Die KI lieferte dabei teilweise fertige Lösungen für das gesamte Projekt. Eher half sie mir, Fehler zu verstehen, neue Konzepte zu lernen und einzelne Probleme Schritt für Schritt selbst zu lösen. Die Vorschläge wurden anschließend von mir angepasst, getestet und in das Projekt integriert.

NOTION

2D Plattformer						
★ Alle Aufgaben	🔄 Nach Status	👤 Meine Aufgaben				
Aufgabenbezeichnung	Status	Verantwortl...	Fälligkeit...	Priorität	Beschreibung	AufwandsL...
Review of Tile Class	Erledigt					
GitHub Repository	Erledigt	Abdullah	05/07/2026	Essenziell		Klein
Tile Class 1	Erledigt	Vincent T.	05/10/2026	Essenziell	Attribute und Konstruktor, da es nur ein S	Klein
Player Class	Erledigt	Abdullah	05/10/2026	Essenziell	Attribute und Methoden Rümpfe impleme	Klein
Enemy Class 1	Erledigt	Vincent T.	05/10/2026	Essenziell	Attribute und Methoden Rümpfe impleme	Klein
Player Movement	Erledigt	Abdullah	05/17/2026	Hoch	Input und simples Movement implementie	Mittel
Rendering auf den Screen	Erledigt	Abdullah	05/20/2026	Essenziell	Rendering auf den Screen durch GameScr	Mittel
World Collision Logik	Erledigt	Abdullah	05/24/2026	Essenziell	Collision Logik implementieren im Zusam	Groß
Player Kamera	Erledigt	Abdullah	05/30/2026	Mittel	Eine Kamera die dem Spieler folgt	Mittel
Player Health System 1	Erledigt	Vincent T.	05/24/2026	Hoch	Health System implementieren mit Health	Groß
Player Attack System	Erledigt	Vincent T.	05/26/2026	Hoch	Ein Attack System mit den benötigten Me	Groß
Ersten Enemy erstellen	Erledigt	Vincent T.	05/31/2026	Hoch	Alle nötigen Methoden implementieren. N	Groß
Enemy Health / Attack System	Erledigt	Vincent T.	06/03/2026	Hoch	Ein Attack und Health System des Enemys	Groß
Player Advanced Movement	Erledigt	Abdullah	06/11/2026	Hoch	Alle Movements für das Spiel implementie	Groß
Überprüfen des Player Movement	Nicht bego...	Vincent T.	06/14/2026	Mittel	Bugs finden und die zuvor Implementierte	Mittel
Heart Threshold anpassen	Erledigt	Abdullah	06/19/2026	Mittel		Mittel
Enemy Logik Verbessern	Erledigt	Abdullah	06/18/2026	Hoch	Springen wenn der Gegner nicht hoch kor	Klein
Attack System Verbessern	Erledigt	Abdullah	07/31/2026	Essenziell		Groß
Game Over Screen	Erledigt	Vincent T.	07/06/2026	Mittel	Todes Screen hinzufügen in GameScreen r	Klein
Main Menu Screen	Erledigt	Vincent T.	07/06/2026	Mittel	Play Button (safe load), Settings, Exit	Mittel

Abb. 3: Zuteilungsübersicht der verschiedenen Aufgaben (erledigt)

Save/Load System	Nicht bego...	Abdullah	07/06/2026	Hoch		Groß
Sprites hinzufügen	Nicht bego...	Abdullah	07/02/2026	Niedrig		Mittel
Settings Funktion	Nicht bego...	Abdullah	07/11/2026	Niedrig		Mittel
spritesheets erstellen	Nicht bego...	Vincent T.	06/19/2026	Mittel	Ein paar sprites erstellen (Seite:https://www	Mittel
Checkpoint System implementieren	Nicht bego...	Abdullah	07/10/2026	Mittel	Checkpoints mithilfe von dem Array einfü	Mittel
Animationen	Nicht bego...	Vincent T.	07/12/2026	Mittel		Mittel
Sound effekts/ Musik	Nicht bego...	Vincent T.	07/17/2026	Niedrig		Mittel
Partikel	Nicht bego...	Abdullah	07/17/2026	Mittel		Groß
Mehr Gegner-Typen	Nicht bego...	Vincent T.	07/24/2026	Hoch		Groß
Erste Zone designen und implementieren	Nicht bego...	Vincent T.	07/31/2026	Mittel	Map in Zones einteilen mit unterschiedlich	Riesig
Boss-Kampf	Nicht bego...	Abdullah	08/03/2026	Hoch		Riesig
NPC und Dialog	Nicht bego...	Vincent T.	09/03/2026	Mittel		Riesig
Cutscenes	Nicht bego...	Vincent T.	08/08/2026	Niedrig		Groß
Items/Power-ups	Nicht bego...	Abdullah	08/08/2026	Mittel		Groß

Abb. 4: Zuteilungsübersicht der verschiedenen Aufgaben (noch offen)

*1: <https://libgdx.com/wiki/graphics/graphics>